



UFR Sciences et Technologies
Département Informatique
M2 Génie Logiciel

* * *

2024-2025

BIG DATA

Pr Marie NDIAYE

Architecture distribuée

Systèmes distribués

systèmes Répartis

Distributed System

Systeme distribue : Definitions

- Des **unités de traitement interconnectées** qui coopèrent pour exécuter des tâches globales en partageant des informations par échanges de messages ou d'événements.
- **Ensemble** composé **d'éléments reliés** par un système de communication; les éléments ont des fonctions de **traitement** (processeurs), de **stockage** (mémoire), de relation avec le monde extérieur (capteurs).
- Un **réseau** composé **d'ordinateurs autonomes** connectés à l'aide d'un middleware de distribution. Ils aident à partager différentes **ressources** et **capacités** pour fournir aux utilisateurs un réseau cohérent unique et intégré.
- Une collection **d'ordinateurs indépendants** qui apparaît à ses utilisateurs comme un seul **système cohérent**. [Tanenbaum]

Caractéristiques (1)

- **Partage des ressources**

- Le partage des ressources signifie que les ressources existantes dans un système distribué sont accessibles à distance par plusieurs ordinateurs du système. Les ordinateurs des systèmes distribués partagent des ressources comme le matériel (disques et imprimantes), les logiciels (fichiers, fenêtres et objets de données) et les données. Les ressources matérielles sont partagées pour réduire le coût et la commodité. Les données sont partagées pour des raisons de cohérence et d'échange d'informations.

- **Réplication / *Replication***

- Les systèmes distribués permettent le partage d'informations et de messages, assurant la cohérence entre des ressources redondantes, telles que des composants logiciels ou matériels, améliorant ainsi la tolérance aux pannes, la fiabilité et l'accessibilité.

Caractéristiques (2)

- **Ouverture**

- L'ouverture concerne les extensions et les améliorations des systèmes distribués. Le système distribué doit être ouvert en termes de matériel et de logiciels.

- **Évolutivité / *Scalability***

- L'évolutivité concerne principalement la façon dont le système distribué gère la croissance lorsque le nombre d'utilisateurs du système augmente. La plupart du temps, nous faisons évoluer le système distribué en ajoutant plus d'ordinateurs dans le réseau. Les composants ne devraient pas avoir besoin d'être modifiés lorsque nous faisons évoluer le système. Les composants doivent être conçus de manière à être évolutifs.

Caractéristiques (3)

- **Concurrence**

- La concurrence est une propriété d'un système représentant le fait que plusieurs activités sont exécutées en même temps. L'exécution simultanée d'activités a lieu dans différents composants tournant sur plusieurs machines dans le cadre d'un système distribué. En outre, ces activités peuvent avoir des interactions entre elles. La concurrence réduit la latence et augmente le débit du système distribué.

- **Tolérance aux pannes / *Availability or fault tolerance***

- Dans un système distribué, le matériel, les logiciels et le réseau peuvent tous tomber en panne. Le système doit être conçu de telle sorte qu'il soit disponible en permanence, même après une défaillance.

Caractéristiques (4)

- **Transparence / *Transparency***

- Les systèmes distribués doivent être perçus par les utilisateurs et les programmeurs d'applications comme un tout plutôt que comme une collection de composants coopérants. La transparence peut être de différents types : accès, localisation, concurrence, réplication, etc.

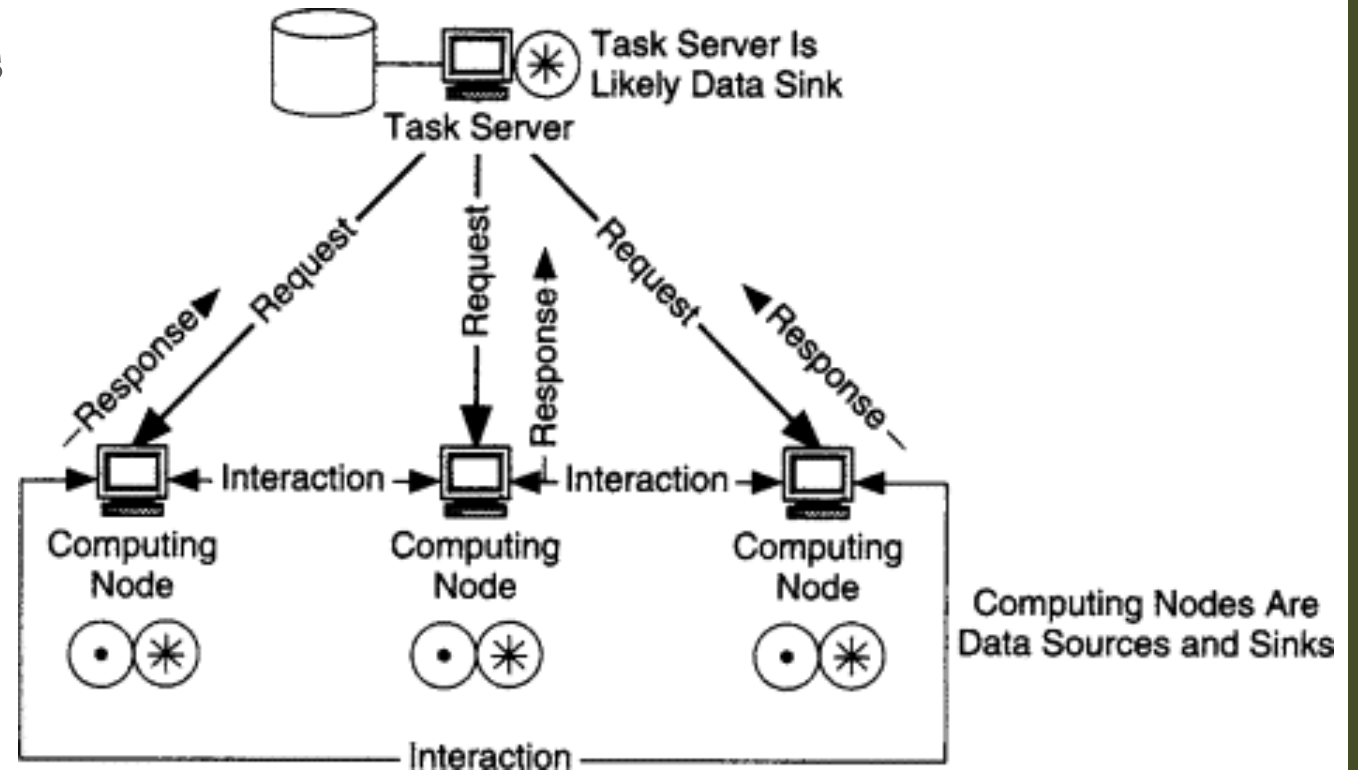
- **Hétérogénéité / *Heterogeneity***

- Dans les systèmes distribués, les composants peuvent présenter une variété et des différences au niveau des réseaux, du matériel informatique, des systèmes d'exploitation, des langages de programmation et des mises en œuvre par différents développeurs.

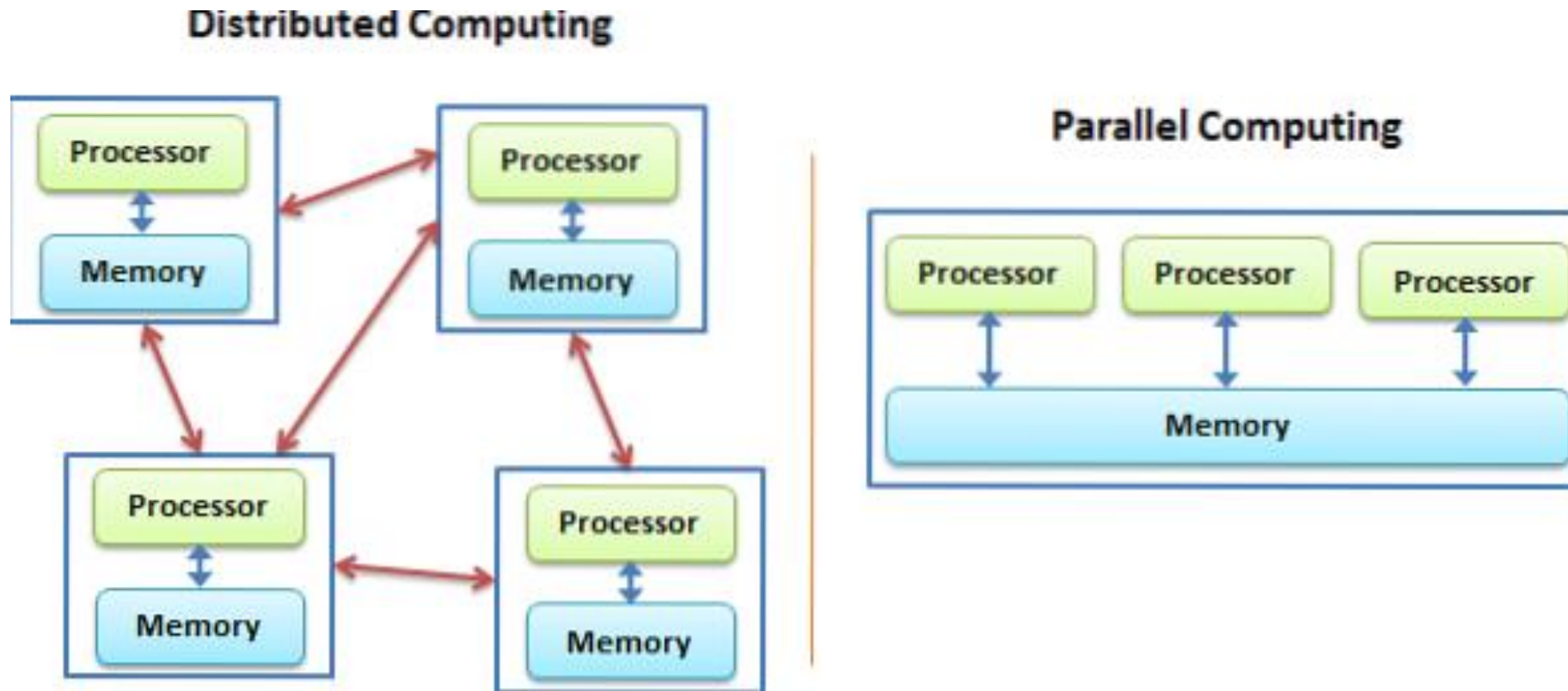
Calcul distribué

Distributed Computing

1. Le calcul distribué consiste à calculer sur des ordinateurs autonomes distribués qui ne communiquent que par l'intermédiaire d'un réseau.
2. Le calcul distribué étudie les modèles, les architectures et les algorithmes utilisés pour construire et gérer des systèmes distribués.



Calcul distribué vs Calcul parallèle



Le framework Apache Hadoop

C'est quoi Hadoop ?

- Description

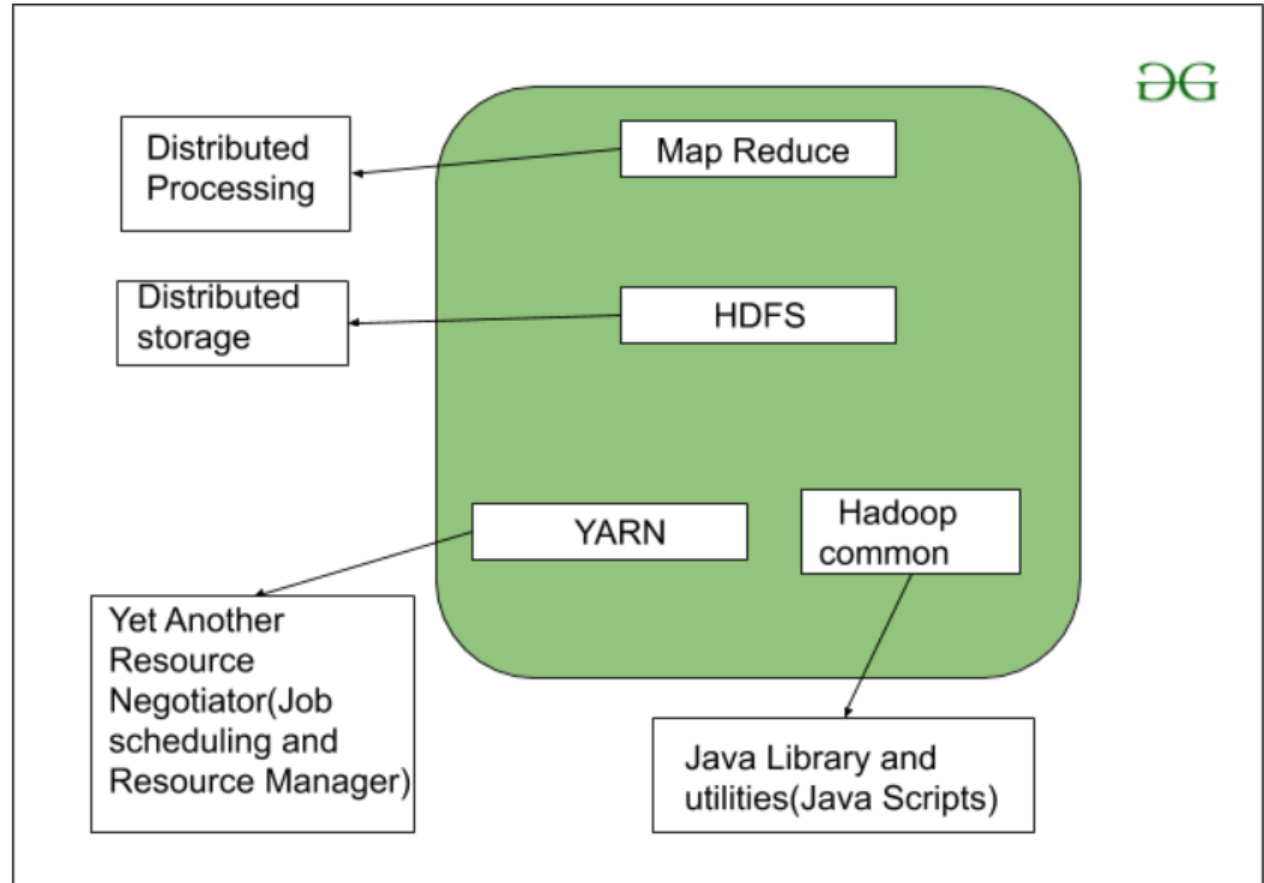
- Hadoop est un framework libre, open source, et écrit en Java permettant le traitement distribué de grands ensembles de données sur des grappes (*clusters*) d'ordinateurs à l'aide de modèles de programmation simples.
- Il est conçu pour passer d'un seul serveur à des milliers de machines, chacune offrant un calcul local et un stockage local.
- Il est destiné à faciliter la création d'applications distribuées (au niveau du **stockage** des données et de leur **traitement**) et échelonnables (scalables).

- Création

- Hadoop a été créé par Doug Cutting (créateur de logiciels open source, architecte en chef de **Cloudera** et président du conseil d'administration de la fondation **Apache** en 2010).
- Sa première version a été publiée en **avril 2006**.
- Il fait partie des projets de la fondation logicielle Apache depuis 2009.
- Son créateur s'est inspiré de la publication de MapReduce, GoogleFS et BigTable de Google
(<https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.454.4747&rep=rep1&type=pdf>) .

L'architecture de Hadoop

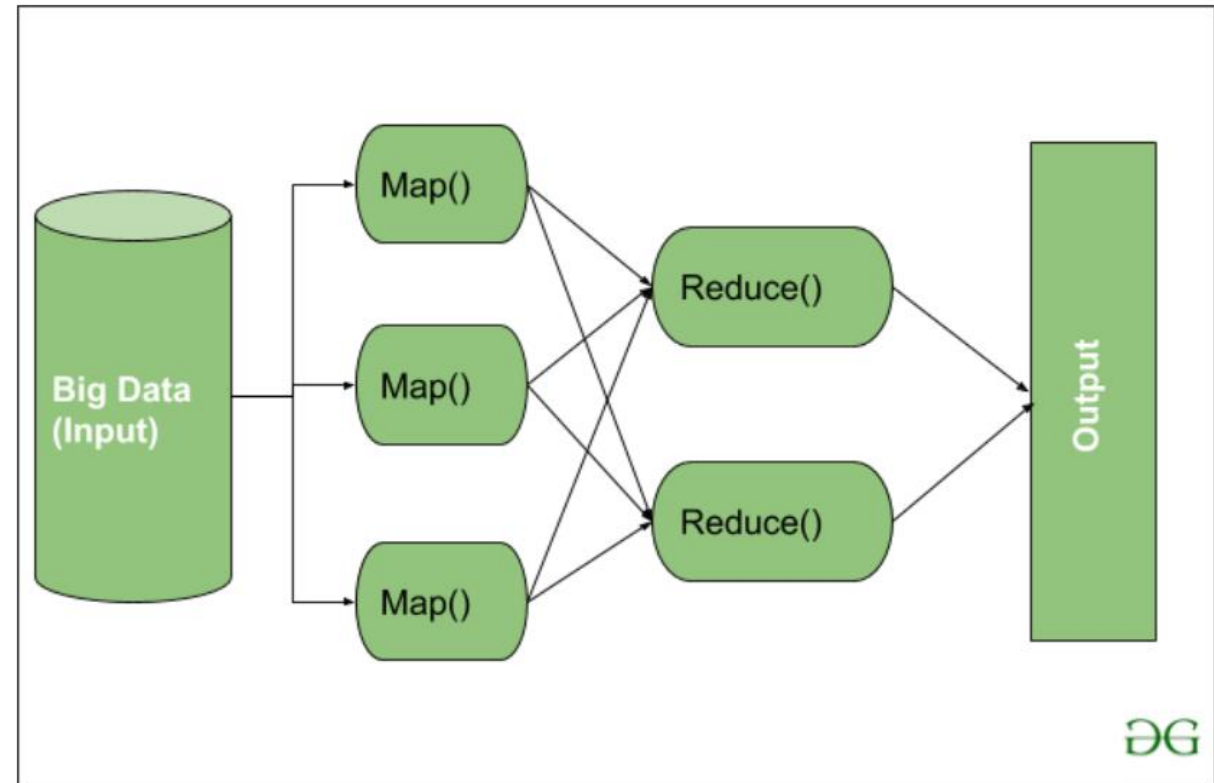
- L'architecture Hadoop se compose principalement de 4 éléments :
 - **MapReduce**
 - **HDFS** (Hadoop distributed File System)
 - **YARN** (Yet Another Resource Framework)
 - Des **utilitaires** Hadoop



Source : <https://www.geeksforgeeks.org/hadoop-architecture/>

Map Reduce

- MapReduce est un algorithme ou une structure de données basée sur le framework YARN.
- La principale caractéristique de MapReduce est d'effectuer le traitement distribué en parallèle dans un cluster Hadoop,
- MapReduce a principalement deux tâches qui sont divisées par phase : Dans la première phase, **Map** est utilisé et dans la phase suivante **Reduce** est utilisé.



Source : <https://www.geeksforgeeks.org/hadoop-architecture/>

YARN

- YARN est un framework sur lequel fonctionne MapReduce.
- Il effectue deux opérations :
 - la planification des tâches
 - L'objectif de la planification des tâches est de diviser une tâche importante en petites tâches afin que chaque tâche puisse être assignée à différents esclaves dans un cluster Hadoop et que le traitement puisse être maximisé.
 - Le planificateur de tâches garde également la trace de la tâche la plus importante, de la tâche la plus prioritaire, des dépendances entre les tâches et de toutes les autres informations comme le calendrier des tâches, etc.
 - la gestion des ressources.
 - Il sert à gérer toutes les ressources qui sont mises à disposition pour faire fonctionner un cluster Hadoop.

Les utilitaires Hadoop (*Hadoop common or Common Utilities*)

- Hadoop Common rassemble les bibliothèques et les utilitaires utilisés par les autres modules.
- Ces utilitaires sont utilisés par HDFS, YARN, et MapReduce pour faire fonctionner le cluster.
- Hadoop Common vérifie les pannes matérielles. En effet, les pannes sont courantes dans un cluster Hadoop et elles doivent être résolues automatiquement.

Le système de fichiers distribué Hadoop

HDFS (Hadoop Distributed File System)

HDFS : Hadoop Distributed File System

- HDFS est un système de fichiers distribué basé sur Java et utilisé dans Hadoop pour stocker une grande quantité de données structurées ou non structurées, d'une taille allant du **gigaoctet** au **pétaoctet**, sur un **cluster** de matériel de base. Il s'agit du stockage le plus fiable connu à ce jour sur la planète.
- Les données sont stockées à plusieurs endroits, de sorte que si l'une des machines tombe en panne, les données peuvent être récupérées sur une autre machine contenant la copie des données. Il est donc hautement tolérant aux pannes et garantit l'absence de perte de données, même en cas de défaillance matérielle.
- Il s'agit du composant principal d'**Hadoop**, avec MapReduce, YARN et d'autres utilitaires courants.
- Il suit une philosophie **Write-Once-Read-Many** qui simplifie la cohérence des données et permet un accès à haut débit.

Notions de Cluster, Rack, Nœud/*Node*

- Hadoop s'exécute sur un cluster d'ordinateurs appelé **cluster Hadoop**
 - Un ***Cluster*** est une collection de racks
 - Un ***rack*** est un ensemble de 30 ou 40 nœuds physiquement stockées et connectés au même commutateur réseau
 - Un **nœud/*Node*** est un ordinateur (serveur)

Cluster

- <https://todorack.online/guia-de-acessorios-para-montar-un-rack/>



Rack

- <https://nexumce.es/producto/armario-rack-19-para-6-baterias-pylontec>

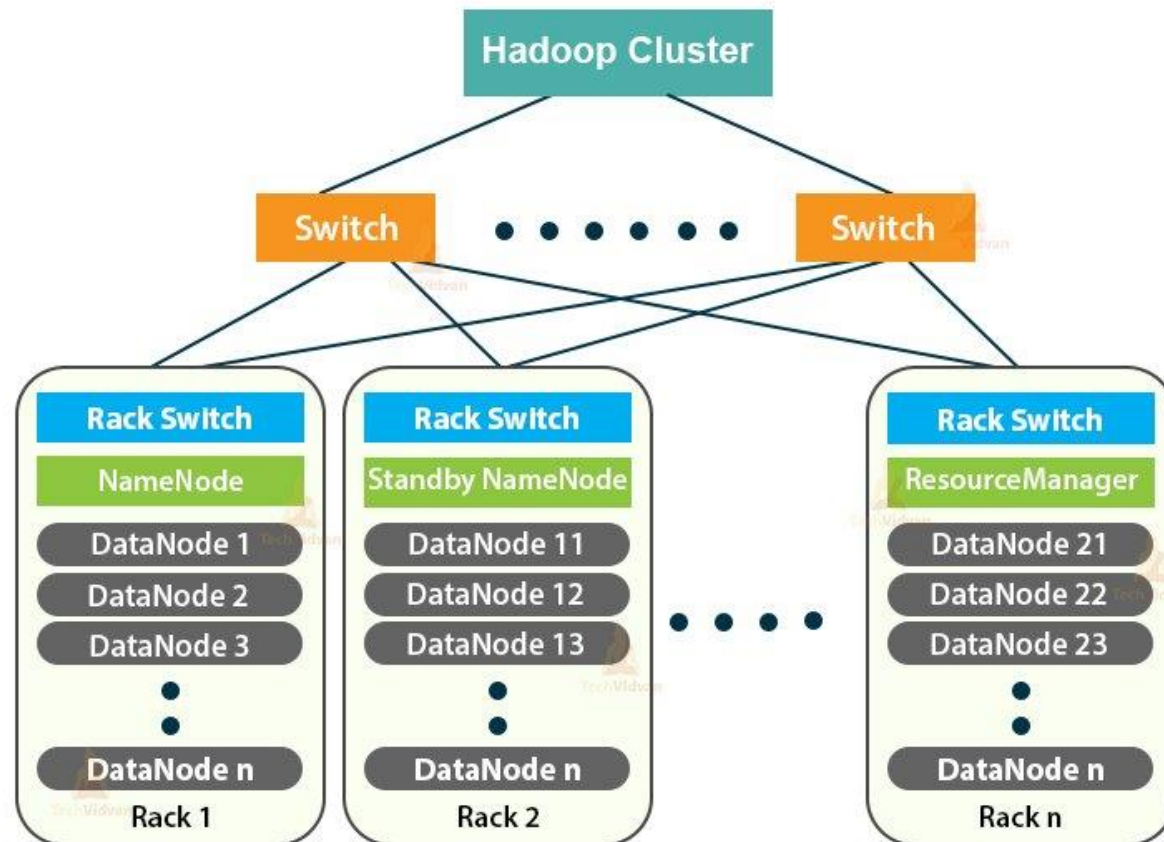


L'architecture de HDFS 1/2

- HDFS suit une architecture **maître-esclave**.
- Le HDFS se compose de deux types de nœuds
 - Le **nœud maître**
 - Il sont la pièce maîtresse de HDFS. Ce sont les machines haut de gamme qui stockent les métadonnées liées à tous les fichiers stockés dans HDFS. Il gère et maintient l'espace de noms du système de fichiers et fournit des instructions aux nœuds esclaves. Le **NameNode** est le nœud maître dans HDFS.
 - Le NameNode gère les opérations de l'espace de nom du système de fichiers comme l'ouverture/la fermeture, le renommage des fichiers et des répertoires. Il fait correspondre les blocs de données aux **DataNodes** et enregistre chaque modification apportée à l'espace de noms du système de fichiers.
 - Les **nœuds esclaves**
 - Ils stockent les données utilisateur et sont responsables du traitement des données en fonction des instructions du nœud maître. Ce sont des machines de configuration normale (matériel de base) qui stockent et traitent les ensembles de données sur instruction du nœud maître. Les **DataNodes** sont les nœuds esclaves dans HDFS.
 - Les DataNodes traitent les demandes de lecture/écriture des clients HDFS. Ils créent, suppriment et répliquent les blocs de données selon les instructions du nœud du NameNode.

L'architecture de HDFS 1/2

- <https://techvidvan.com/tutorials/hadoop-cluster/>

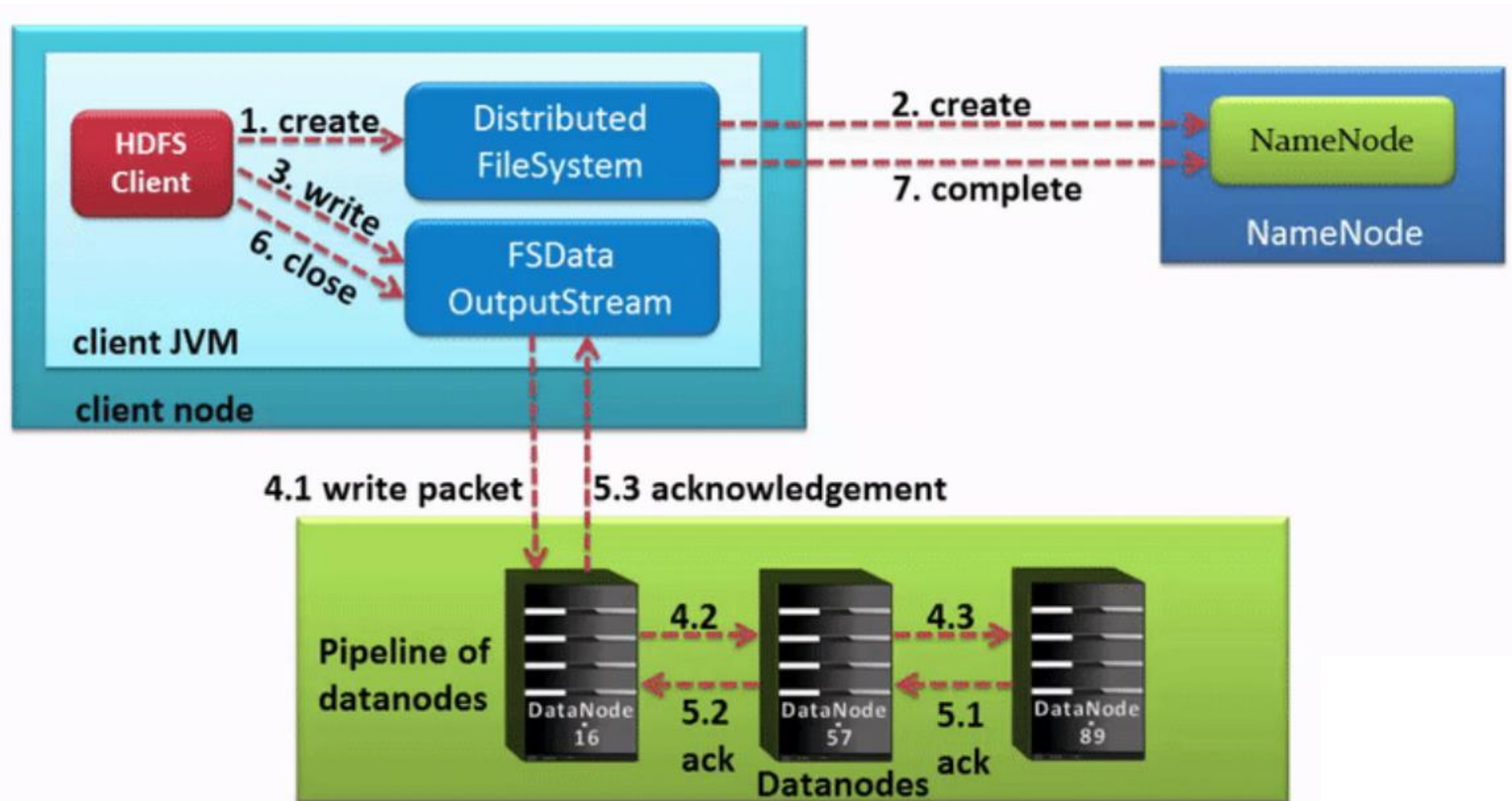


Fonctionnement de HDFS

Ecriture

Lecture

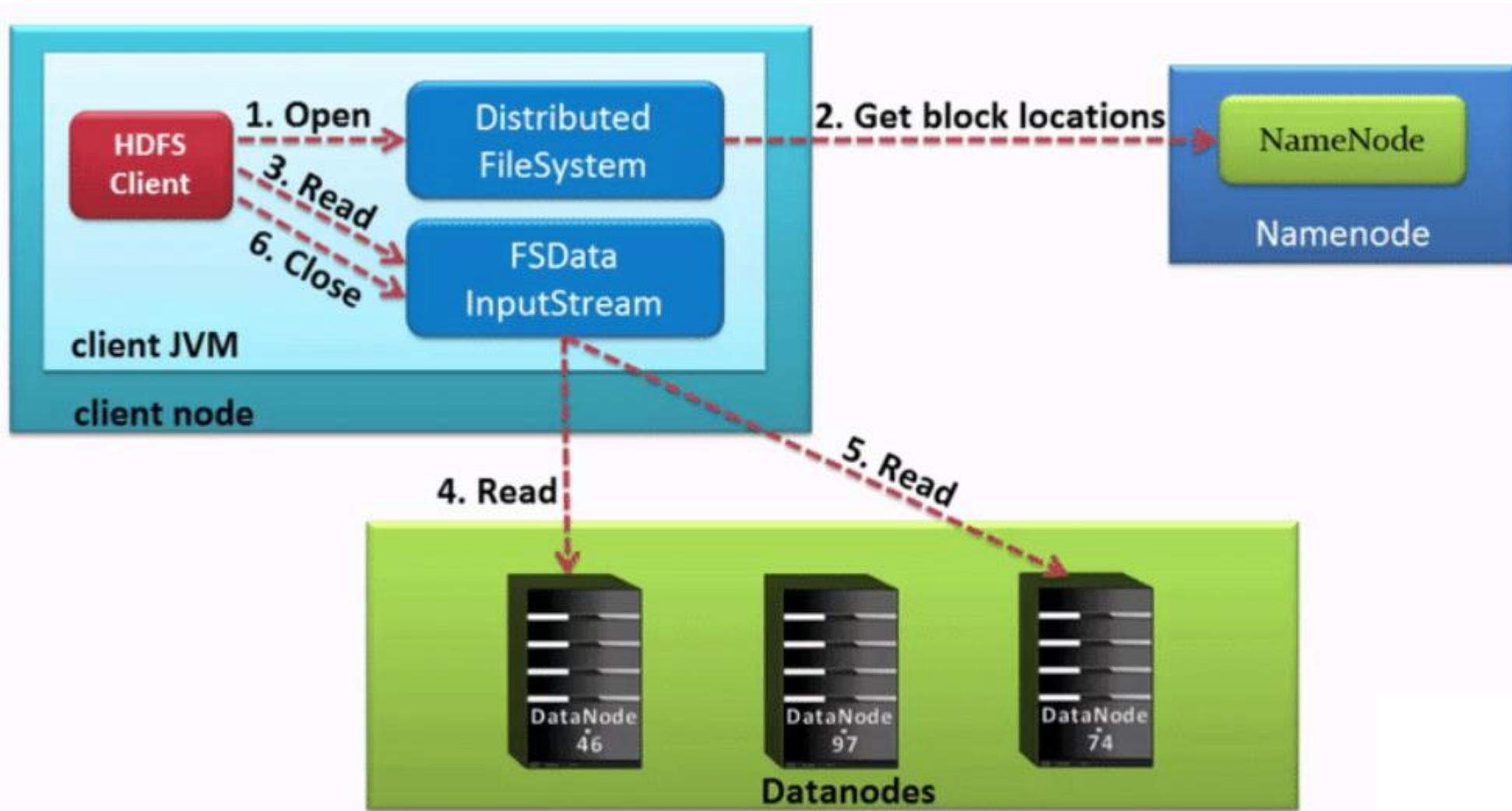
Ecriture



Exemple de code d'écriture d'un fichier dans HDFS en Java

```
1.  FileSystem fileSystem = FileSystem.get(conf);
2.  // Check if the file already exists
3.  Path path = new Path("/path/to/file.ext");
4.  if (fileSystem.exists(path)) {
5.      System.out.println("File " + dest + " already exists");
6.      return;
7.  }
8.  // Create a new file and write data to it.
9.  FSDataOutputStream out = fileSystem.create(path);
10. InputStream in = new BufferedInputStream(new FileInputStream(
11.     new File(source)));
12. byte[] b = new byte[1024];
13. int numBytes = 0;
14. while ((numBytes = in.read(b)) > 0) {
15.     out.write(b, 0, numBytes);
16. }
17. // Close all the file descriptors
18. in.close();
19. out.close();
20. fileSystem.close();
```

Lecture



Exemple de code de lecture d'un fichier dans HDFS en Java

```
1.  FileSystem fileSystem = FileSystem.get(conf);
2.  Path path = new Path("/path/to/file.ext");
3.  if (!fileSystem.exists(path)) {
4.      System.out.println("File does not exists");
5.      return;
6.  }
7.  FSDataInputStream in = fileSystem.open(path);
8.  int numBytes = 0;
9.  while ((numBytes = in.read(b)) > 0) {
10.     System.out.println((char)numBytes); // code to manipulate the data which is read
11. }
12. in.close();
13. out.close();
14. fileSystem.close();
```

Références

- **Systèmes distribués**

- <https://fr.theastrologypage.com/distributed-system>
- <http://www.ousmanethiare.com/images/cours/introductionauxsd.pdf>
- <https://fr.acervolima.com/caracteristiques-du-systeme-d-exploitation-distribue/>
- <https://medium.com/system-design-blog/key-characteristics-of-distributed-systems-781c4d92cce3>
- Vidéo → <https://www.youtube.com/watch?v=ajjOEltiZm4>

- **Calcul distribué**

- <https://www.youtube.com/watch?v=ajjOEltiZm4>
- <https://www.sciencedirect.com/topics/computer-science/distributed-computing>
- <https://www.ridge.co/blog/what-is-distributed-computing/>
- https://fr.differbetween.com/article/difference_between_distributed_computing_and_parallel_computing

- **Hadoop**

- <https://hadoop.apache.org/>
- <https://www.lebigdata.fr/hadoop>
- <https://www.geeksforgeeks.org/hadoop-architecture/>